HackNEX — API Design Document

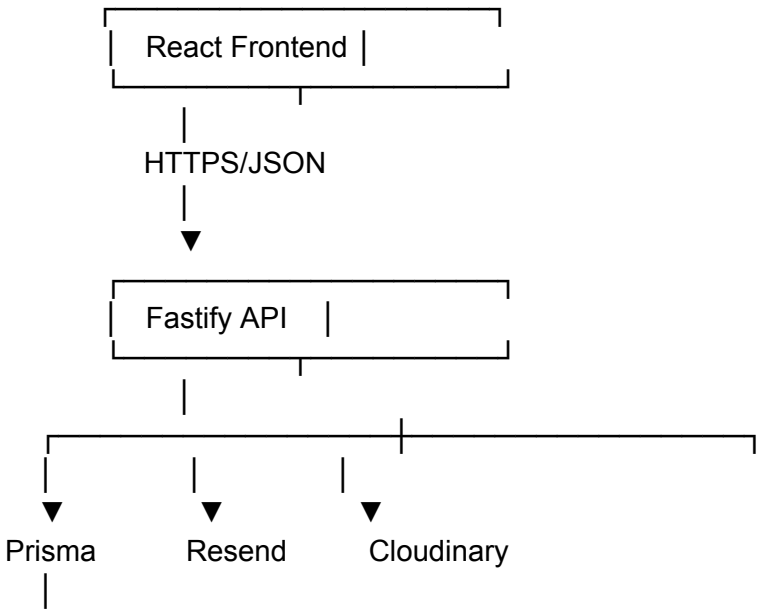
Document Version: 1.0
Project: HackNEX
Organizer: NEXUS Club, Karunya Institute of Technology and Sciences
Event: October 7–9, 2026
Registration: September 8–October 1, 2026
Expected Participants: 1,500+
Team Size: Exactly 4
Registration Fee: ₹600/team
Backend: Node.js + Fastify + TypeScript
ORM: Prisma
Database: PostgreSQL
Authentication: Email + Password + Email Verification
Email: Resend
Media: Cloudinary

> **Payment integration note:** Karunya's exact payment API/webhook specification is currently unavailable. The API below therefore defines a provider-independent payment layer, with the Karunya adapter to be finalized once the official payment API details are available.

---

1. API Architecture

The HackNEX application follows:



▼
PostgreSQL

External payment:

React
↓
Fastify
↓
Karunya Payment System
↓
Payment Result/Webhook
↓
Fastify
↓
PostgreSQL
↓
Resend
↓
Confirmation Email

---

2. API Base URL

Development:

http://localhost:4000/api

Production:

https://<backend-domain>/api

The actual production backend domain will be decided during deployment.

---

3. API Versioning

Use:

/api/v1

Therefore production endpoints become:

/api/v1/auth/login
/api/v1/registrations

/api/v1/payments

This allows a future:

/api/v2

without breaking the existing frontend.

---

4. HTTP Methods

Method	Purpose
GET	Retrieve data
POST	Create data/action
PATCH	Partially update
PUT	Replace resource
DELETE	Delete/deactivate resource

---

5. Standard API Response

Successful response:

```
{
  "success": true,
  "data": {},
  "message": "Request successful"
}
```

Error:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid request",
    "details": []
  }
}
```

---

6. HTTP Status Codes

The API will use standard HTTP status codes.

Status	Meaning
200	Successful request
201	Resource created
204	Successful request, no content
400	Bad request
401	Not authenticated
403	Not authorized
404	Resource not found
409	Conflict
422	Validation error
429	Rate limit exceeded
500	Internal server error
502	External service failure
503	Service unavailable

---

7. Authentication Architecture

HackNEX uses:

Email
+
Password
+
Email Verification
+
Session

No:

- Google login
- GitHub login
- Social login

---

8. Authentication Endpoints

POST /api/v1/auth/signup
POST /api/v1/auth/login
POST /api/v1/auth/logout
GET /api/v1/auth/me
GET /api/v1/auth/verify-email
POST /api/v1/auth/resend-verification
POST /api/v1/auth/forgot-password
POST /api/v1/auth/reset-password

---

9. Signup

Endpoint

POST /api/v1/auth/signup

Request

```
{
  "name": "Infant Bosco",
  "email": "infant@example.com",
  "password": "********"
}
```

Validation

name:
  required
  2–150 characters

email:
  required
  valid email
  normalized to lowercase

password:
  required
  minimum strength requirement

Response

```
{
  "success": true,
  "message": "Account created. Please verify your email."
}
```

The system sends:

Email Verification
↓
Resend

---

10. Login

Endpoint

POST /api/v1/auth/login

Request

```
{
  "email": "infant@example.com",
  "password": "********"
}
```

Rules

If email isn't verified:

LOGIN → REJECT

Response:

```
{
  "success": false,
  "error": {
    "code": "EMAIL_NOT_VERIFIED",
    "message": "Please verify your email before logging in."
  }
}
```

---

11. Get Current User

Endpoint

GET /api/v1/auth/me

Authentication:

Required

Response:

```
{
  "success": true,
  "data": {
    "id": "...",
    "name": "Infant Bosco",
    "email": "infant@example.com",
    "emailVerified": true
  }
}
```

---

12. Logout

POST /api/v1/auth/logout

The current session is invalidated.

---

13. Email Verification

Endpoint

GET /api/v1/auth/verify-email?token=<token>

Process:

Token
↓
Validate
↓
Check expiry
↓
Mark User.emailVerified = true
↓
Invalidate token

---

14. Resend Verification

POST /api/v1/auth/resend-verification

Request:

```
{
  "email": "infant@example.com"
}
```

Rate-limited to prevent abuse.

---

15. Forgot Password

POST /api/v1/auth/forgot-password

Request:

```
{
  "email": "infant@example.com"
}
```

The system sends a password-reset email.

---

16. Reset Password

POST /api/v1/auth/reset-password

Request:

```
{
  "token": "...",
  "password": "********"
}
```

---

17. Public Website APIs

The landing page contains dynamic content.

Recommended endpoints:

GET /api/v1/domains
GET /api/v1/schedule
GET /api/v1/prizes
GET /api/v1/sponsors
GET /api/v1/faqs
GET /api/v1/announcements
GET /api/v1/media

This allows administrators to update content without modifying the React application.

---

18. Domains

Get Domains

GET /api/v1/domains

Response:


```
{
  "success": true,
  "data": [
    {
      "id": "...",
      "name": "Generative AI",
      "slug": "generative-ai",
      "description": "...",
      "icon": "...",
      "displayOrder": 1
    }
  ]
}
```


---

19. Schedule

GET /api/v1/schedule

Optional:

?day=2026-10-07

Response:


```
{
  "success": true,
```

```
  "data": [
    {
      "id": "...",
      "title": "Opening Ceremony",
      "description": "...",
      "startTime": "...",
      "endTime": "...",
      "location": "KITS",
      "day": "October 7"
    }
  ]
}
```


---

20. Prizes

GET /api/v1/prizes

Response:


```
{
  "success": true,
  "data": [
    {
      "title": "First Prize",
      "description": "...",
      "amount": 100000,
      "position": 1
    }
  ]
}
```


---

21. Sponsors

GET /api/v1/sponsors

Returns:

Sponsor
Logo
Website
Tier
Display Order

---

22. FAQ

GET /api/v1/faqs

Response:

```
{
  "success": true,
  "data": [
    {
      "question": "Who can participate?",
      "answer": "...",
      "displayOrder": 1
    }
  ]
}
```

---

23. Media

GET /api/v1/media

Optional:

?category=HERO

Response:

```
{
  "success": true,
  "data": [
    {
      "id": "...",
      "secureUrl": "https://...",
      "category": "HERO",
      "altText": "HackNEX",
      "displayOrder": 1
    }
  ]
}
```

---

24. Registration APIs

This is the most important API module.

POST /api/v1/registrations
GET /api/v1/registrations/me
GET /api/v1/registrations/:id
PATCH /api/v1/registrations/:id
POST /api/v1/registrations/:id/submit

Only authenticated and verified users can access these.

---

25. Registration Creation

Endpoint

POST /api/v1/registrations

Authentication:

Required

The authenticated user automatically becomes the captain.

---

26. Registration Request


```
{
  "teamName": "Code Titans",
  "participants": [
    {
      "name": "Captain Name",
      "email": "captain@example.com",
      "phone": "9876543210",
      "college": "Karunya Institute",
      "department": "CSE",
      "yearOfStudy": "2",
      "linkedinUrl": "https://linkedin.com/in/example",
      "foodPreference": "VEG"
    },
    {
      "name": "Member Two",
      "email": "member2@example.com",
      "phone": "9876543211",
      "college": "PSG College",
      "department": "ECE",
      "yearOfStudy": "3",
```

```
    "linkedinUrl": "https://linkedin.com/in/example2",
    "foodPreference": "NON_VEG"
   },
   {
    "name": "Member Three",
    "email": "member3@example.com",
    "phone": "9876543212",
    "college": "CIT",
    "department": "IT",
    "yearOfStudy": "2",
    "linkedinUrl": "https://linkedin.com/in/example3",
    "foodPreference": "VEG"
   },
   {
    "name": "Member Four",
    "email": "member4@example.com",
    "phone": "9876543213",
    "college": "Other College",
    "department": "CSE",
    "yearOfStudy": "1",
    "linkedinUrl": "https://linkedin.com/in/example4",
    "foodPreference": "NON_VEG"
   }
  ]
}
```


---

27. Registration Validation

The backend validates:

Exactly 4 participants
Exactly 1 captain
Unique participant emails
Valid emails
Valid phone numbers
Valid LinkedIn URLs
Valid food preferences
Non-empty college
Non-empty department
Valid year of study
Unique team name
Registration period
Authenticated captain
Verified captain email

---

28. Registration Creation Transaction

The backend executes:

BEGIN TRANSACTION

Check authentication
↓
Check email verification
↓
Check registration deadline
↓
Check team name
↓
Check participant uniqueness
↓
Create Team
↓
Create 4 Participants
↓
Create Registration
↓
COMMIT

If anything fails:

ROLLBACK

---

29. Registration Response

```
{
  "success": true,
  "data": {
    "registrationId": "HNX-2026-000001",
    "teamId": "...",
    "status": "READY_FOR_PAYMENT",
    "amount": 600,
    "currency": "INR"
  }
}
```

---

30. Get My Registration

GET /api/v1/registrations/me

Authentication:

Required

Response:

```
{
  "success": true,
  "data": {
    "registrationId": "HNX-2026-000001",
    "teamName": "Code Titans",
    "status": "PAYMENT_PENDING",
    "participants": []
  }
}
```

---

31. Important Registration Rule

The captain can only register **one team**.

Therefore:

User
↓
Existing Team?
↓
YES → Reject new registration
NO → Allow

This is also protected by:

Team.captainUserId UNIQUE

---

32. Registration Deadline

The backend must enforce:

Registration Start:
September 8, 2026

Registration End:
October 1, 2026

The frontend countdown is only informational.

The backend is the final authority.

---

33. Update Registration

PATCH /api/v1/registrations/:id

Allowed only while registration is not confirmed.

For example:

DRAFT
READY_FOR_PAYMENT
PAYMENT_PENDING

should have carefully controlled edit behavior.

Once payment is verified:

CONFIRMED

participant details should generally become locked.

---

34. Submit Registration

POST /api/v1/registrations/:id/submit

Process:

Validate all data
↓
Validate exactly 4 members
↓
Validate team
↓
Create payment
↓
Registration → PAYMENT_PENDING
↓
Return payment initiation information

---

35. Payment APIs

POST /api/v1/payments/create
GET /api/v1/payments/:id
POST /api/v1/payments/webhook/karunya

---

36. Create Payment

POST /api/v1/payments/create

Request:

```
{
  "registrationId": "HNX-2026-000001"
}
```

The backend determines:

Amount = ₹600
Currency = INR
Provider = KARUNYA

The client cannot modify the amount.

---

37. Payment Creation Response

Provider-dependent:

```
{
  "success": true,
  "data": {
    "paymentId": "...",
    "amount": 600,
    "currency": "INR",
    "provider": "KARUNYA",
    "paymentUrl": "..."
  }
}
```

If Karunya provides an embedded payment mechanism instead of a URL, this response will be adapted.

---

38. Payment Status

GET /api/v1/payments/:id

Response:

```
{
  "success": true,
  "data": {
    "paymentId": "...",
    "status": "PROCESSING",
    "amount": 600,
    "currency": "INR"
  }
}
```

---

39. Karunya Webhook

POST /api/v1/payments/webhook/karunya

This endpoint is called by the Karunya payment system.

Exact request format: TBD.

Example conceptual payload:

```
{
  "transactionId": "...",
  "providerReference": "...",
  "amount": 600,
  "status": "SUCCESS",
  "signature": "..."
}
```

---

40. Payment Webhook Security

The webhook must not simply trust:

```
{
  "status": "SUCCESS"
}
```

The backend must verify the payment using whatever mechanism Karunya provides:

Webhook
↓
Verify Signature
↓
Verify Transaction
↓
Verify Amount
↓
Verify Registration
↓
Update Payment

---

41. Payment Verification Transaction

BEGIN

Check transaction ID

Check payment amount = ₹600

Check payment hasn't already been processed

Update Payment → VERIFIED

Update Registration → PAYMENT_VERIFIED

Update Team → CONFIRMED

Generate confirmation

COMMIT

---

42. Confirmation Email Trigger

Only after successful database confirmation:

Payment VERIFIED
↓
Registration CONFIRMED
↓
Resend
↓
Registration Confirmation Email

This is critical.

The frontend must **never** trigger the confirmation email.

---

43. Payment Failure

If payment fails:

Payment → FAILED
Registration → PAYMENT_PENDING

The captain can retry payment.

The participant information remains stored.

---

44. No Refund Policy

The API should not provide a normal refund endpoint.

There will be:

NO /payments/refund

The no-refund policy is a business rule.

If organizers later need administrative cancellation/refund handling, it should be a restricted admin operation rather than a public API.

---

45. Admin API Architecture

All admin endpoints require:

Authenticated User
↓
Admin record
↓
isActive = true
↓
Allow

---

46. Admin Endpoints

GET /api/v1/admin/dashboard

GET /api/v1/admin/teams
GET /api/v1/admin/teams/:id
PATCH /api/v1/admin/teams/:id

GET /api/v1/admin/participants
GET /api/v1/admin/participants/:id

GET /api/v1/admin/payments
GET /api/v1/admin/payments/:id

GET /api/v1/admin/registrations
GET /api/v1/admin/registrations/:id

---

47. Admin Dashboard

GET /api/v1/admin/dashboard

Response can include:

```
{
  "success": true,
  "data": {
    "totalTeams": 375,
    "confirmedTeams": 350,
    "pendingPayments": 20,
    "failedPayments": 5,
    "totalParticipants": 1400,
    "verifiedRevenue": 210000
  }
}
```

These figures must be calculated from the database.

---

48. Admin Team List

GET /api/v1/admin/teams

Pagination:

?page=1&limit=25

Filtering:

?status=CONFIRMED

Search:

?search=Code

Sorting:

?sortBy=createdAt&order=desc

---

49. Admin Participant List

GET /api/v1/admin/participants

Filters:

college
department
yearOfStudy
foodPreference
team

Example:

/api/v1/admin/participants
?college=Karunya
&foodPreference=VEG
&page=1
&limit=25

---

50. Admin Payment List

GET /api/v1/admin/payments

Filters:

status
date
team
transactionId

---

51. Admin Registration Lookup

GET /api/v1/admin/registrations/:id

Returns:

Registration
Team
Captain
All Participants
Payment
Registration Status
Payment Status
Timestamps

---

52. Admin Content APIs

Domains

POST /api/v1/admin/domains
PATCH /api/v1/admin/domains/:id
DELETE /api/v1/admin/domains/:id

Schedule

POST /api/v1/admin/schedule
PATCH /api/v1/admin/schedule/:id
DELETE /api/v1/admin/schedule/:id

Prizes

POST /api/v1/admin/prizes
PATCH /api/v1/admin/prizes/:id
DELETE /api/v1/admin/prizes/:id

Sponsors

POST /api/v1/admin/sponsors
PATCH /api/v1/admin/sponsors/:id
DELETE /api/v1/admin/sponsors/:id

FAQs

POST /api/v1/admin/faqs
PATCH /api/v1/admin/faqs/:id

DELETE /api/v1/admin/faqs/:id

Announcements

POST /api/v1/admin/announcements
PATCH /api/v1/admin/announcements/:id
DELETE /api/v1/admin/announcements/:id
POST /api/v1/admin/announcements/:id/publish

---

53. Cloudinary API

Media management:

POST /api/v1/admin/media
DELETE /api/v1/admin/media/:id
PATCH /api/v1/admin/media/:id
GET /api/v1/media

Recommended upload architecture:

Admin
↓
Fastify
↓
Generate Cloudinary upload signature
↓
React
↓
Cloudinary
↓
Return public_id + secure_url
↓
Fastify
↓
PostgreSQL

This avoids unnecessarily routing large image files through the backend.

---

54. Email API Architecture

The backend will integrate with Resend.

The frontend never communicates directly with Resend.

Fastify
↓
Email Service
↓
Resend

Email events:

SIGNUP
↓
Verification Email

FORGOT PASSWORD
↓
Reset Email

PAYMENT VERIFIED
↓
Registration Confirmation

ANNOUNCEMENT
↓
Announcement Email

---

55. Internal Email Service

Instead of calling Resend throughout the codebase, create:

email.service.ts

Functions:

sendVerificationEmail()
sendPasswordResetEmail()
sendRegistrationConfirmation()
sendAnnouncementEmail()

This makes the email provider replaceable.

---

56. Announcement Email Architecture

When an admin publishes an announcement:

Admin
↓
Publish Announcement
↓
Database
↓
Email Queue
↓
Registered Participants
↓
Resend

For 1,500+ participants, emails should **not** be sent in one synchronous HTTP request.

Use a background job/queue architecture.

---

57. Recommended Background Jobs

Potential jobs:

send-verification-email
send-password-reset-email
send-registration-confirmation
send-announcement
payment-reconciliation
cleanup-expired-sessions

---

58. Rate Limiting

Rate limiting is required for:

/signup
/login
/resend-verification
/forgot-password
/payment/create

Especially:

Login
Signup
Password reset

to prevent abuse.

---

59. API Authentication

Protected API:

Authorization
↓
Session
↓
User

The frontend should not be trusted to provide:

userId
adminId
captainUserId

The backend derives the authenticated user from the session.

---

60. Authorization Matrix

API	Public	User	Admin
Public content	✅	✅	✅
Signup	✅	—	—
Login	✅	—	—
Registration	❌	✅	—
Own registration	❌	✅	—
All registrations	❌	❌	✅
Payments	❌	Own	All
Content management	❌	❌	✅
Media management	❌	❌	✅

Dashboard	❌	❌	✅

---

61. API Validation

Use a schema validation library such as:

Zod

Every API request should have a schema.

Example conceptually:

```
POST /registrations
      ↓
Zod Schema
      ↓
Valid?
      ┌──────────┴──────────┐
NO          YES
↓           ↓
422         Service
```

---

62. Error Codes

Standardized application error codes:

AUTH_REQUIRED
INVALID_CREDENTIALS
EMAIL_NOT_VERIFIED
EMAIL_ALREADY_EXISTS
INVALID_VERIFICATION_TOKEN
TOKEN_EXPIRED

TEAM_NAME_EXISTS
TEAM_NOT_FOUND
TEAM_ALREADY_REGISTERED
INVALID_TEAM_SIZE
DUPLICATE_PARTICIPANT

REGISTRATION_NOT_FOUND
REGISTRATION_CLOSED
REGISTRATION_ALREADY_CONFIRMED

PAYMENT_NOT_FOUND
PAYMENT_FAILED
PAYMENT_ALREADY_VERIFIED
PAYMENT_AMOUNT_MISMATCH
PAYMENT_VERIFICATION_FAILED

ADMIN_REQUIRED
ADMIN_INACTIVE

VALIDATION_ERROR
RATE_LIMIT_EXCEEDED
INTERNAL_SERVER_ERROR

---

63. Pagination Standard

All potentially large collections use pagination.

Example:

GET /api/v1/admin/participants?page=2&limit=25

Response:

```
{
  "success": true,
  "data": [],
  "pagination": {
    "page": 2,
    "limit": 25,
    "total": 1500,
    "totalPages": 60
  }
}
```

Never return thousands of participant records in a single normal dashboard request.

---

64. Search

Admin search should support:

Team Name
Registration ID

Participant Name
Participant Email
Transaction ID
College

Example:

GET /api/v1/admin/teams?search=HNX-2026-000001

---

65. API Security

The API should implement:

HTTPS
CORS
Rate Limiting
Input Validation
Authentication
Authorization
Secure Cookies
Password Hashing
CSRF Protection where applicable
Webhook Signature Verification
SQL Injection Protection through Prisma
Security Headers
Request Size Limits

---

66. CORS

Only the official HackNEX frontend origin should be allowed in production.

Example:

Frontend
https://hacknex-domain.com

Backend
https://api.hacknex-domain.com

Exact domains will be configured later.

---

67. Logging

Backend logs should record:

Request ID
HTTP method
Endpoint
Status code
Response time
Error code

But **never** log:

Passwords
Session tokens
Payment secrets
API keys
Sensitive payment payloads

---

68. Monitoring

Sentry will monitor:

Frontend errors
Backend errors
Unhandled exceptions
API failures
Payment integration failures
Registration failures

---

69. Health Check

Endpoint

GET /api/v1/health

Response:


```
{
  "success": true,
  "data": {
    "status": "healthy",
    "database": "healthy"
```

```
  }
}
```

A deeper readiness endpoint can be:

GET /api/v1/health/ready

---

70. API Request Flow — Normal Registration

Captain
↓
Login
↓
Email Verified?
↓
YES
↓
Registration Form
↓
POST /registrations
↓
Validate
↓
Create Team
↓
Create Participants
↓
Create Registration
↓
READY_FOR_PAYMENT
↓
POST /payments/create
↓
Karunya
↓
Payment

---

71. API Request Flow — Successful Payment

Karunya
↓
Webhook
↓
POST /payments/webhook/karunya
↓
Verify Signature
↓
Verify Transaction
↓
Verify Amount
↓
Payment = VERIFIED
↓
Registration = CONFIRMED
↓
Team = CONFIRMED
↓
Generate Confirmation
↓
Email Queue
↓
Resend
↓
Captain Email

---

72. API Request Flow — Failed Payment

Karunya
↓
Payment FAILED
↓
Webhook
↓
Payment = FAILED
↓
Registration remains
PAYMENT_PENDING
↓
Captain retries payment

---

73. API Request Flow — Duplicate Participant

Registration
↓
Participant Email
↓
Check Database
↓
Already exists
↓
409 CONFLICT

Response:

```
{
  "success": false,
  "error": {
    "code": "DUPLICATE_PARTICIPANT",
    "message": "A participant with this email is already registered."
  }
}
```

---

74. API Request Flow — Registration Closed

Even if the frontend still displays the registration button:

POST /registrations
↓
Backend checks date
↓
October 1 passed
↓
Reject

Response:

403 / 422
REGISTRATION_CLOSED

The backend remains the authoritative source.

---

75. API Module Structure

The backend should be organized approximately as:

```
src/
│
├── app.ts
├── server.ts
│
├── config/
│
├── plugins/
│
├── modules/
│   │
│   ├── auth/
│   │   ├── auth.routes.ts
│   │   ├── auth.service.ts
│   │   └── auth.schema.ts
│   │
│   ├── registration/
│   │   ├── registration.routes.ts
│   │   ├── registration.service.ts
│   │   └── registration.schema.ts
│   │
│   ├── payment/
│   │   ├── payment.routes.ts
│   │   ├── payment.service.ts
│   │   ├── payment.schema.ts
│   │   └── providers/
│   │       └── karunya.provider.ts
│   │
│   ├── admin/
│   │
│   ├── domains/
│   ├── schedule/
│   ├── prizes/
│   ├── sponsors/
│   ├── faq/
│   ├── announcements/
│   └── media/
│
├── services/
│   ├── email.service.ts
│   ├── cloudinary.service.ts
│   └── ...
│
```

```
├── middleware/
│
└── utils/
```

---

76. Payment Provider Abstraction

This is particularly important because the Karunya integration isn't finalized.

Instead of building:

```
Registration
    ↓
Karunya-specific code everywhere
```

use:

```
PaymentService
     │
     └── PaymentProvider
           │
           └── KarunyaProvider
```

Conceptually:

```
interface PaymentProvider {

  createPayment()

  verifyPayment()

  handleWebhook()

}
```

Then:

KarunyaProvider

implements it.

If Karunya changes its payment mechanism, the rest of the application doesn't need to be rewritten.

---

77. API Documentation

The backend should expose OpenAPI documentation.

Development:

/api/docs

The documentation should describe:

- Endpoint
- Method
- Authentication
- Request
- Response
- Status codes
- Errors
- Schemas

This will make frontend development much easier.

---

78. API Testing

Every important endpoint should have automated tests.

Authentication

Signup
Login
Logout
Email verification
Password reset

Registration

Valid registration
3 members
5 members
Duplicate email
Duplicate team
Unverified captain
Registration closed

Payment

Payment creation

Successful webhook
Failed webhook
Duplicate webhook
Invalid signature
Wrong amount

Admin

Unauthorized admin
Admin access
Pagination
Filtering
Content management

---

79. API Test Environment

Use:

Development Database
↓
Test Database
↓
Automated API Tests

Do not run destructive tests against production.

---

80. Final API Endpoint Map

AUTH
├── POST /auth/signup
├── POST /auth/login
├── POST /auth/logout
├── GET /auth/me
├── GET /auth/verify-email
├── POST /auth/resend-verification
├── POST /auth/forgot-password
└── POST /auth/reset-password

PUBLIC CONTENT
├── GET /domains
├── GET /schedule
├── GET /prizes

```
├── GET /sponsors
├── GET /faqs
├── GET /announcements
└── GET /media


REGISTRATION
├── POST  /registrations
├── GET   /registrations/me
├── GET   /registrations/:id
├── PATCH /registrations/:id
└── POST  /registrations/:id/submit


PAYMENT
├── POST /payments/create
├── GET  /payments/:id
└── POST /payments/webhook/karunya


ADMIN
├── GET   /admin/dashboard
├── GET   /admin/teams
├── GET   /admin/teams/:id
├── PATCH /admin/teams/:id
├── GET   /admin/participants
├── GET   /admin/participants/:id
├── GET   /admin/registrations
├── GET   /admin/registrations/:id
├── GET   /admin/payments
└── GET   /admin/payments/:id


ADMIN CONTENT
├── Domains
├── Schedule
├── Prizes
├── Sponsors
├── FAQs
├── Announcements
└── Media


SYSTEM
├── GET /health
└── GET /health/ready
```

---

81. Final API Architecture

The complete backend flow is therefore: